

How Roognis AI Works

Plain English guide for the team · What happens step by step for every user action

Services

1. Login

2. Student Chat

3. Image

4. Video

5. Feedback

6. Teacher Dashboard

7. Parent Dashboard

8. PDF Upload

What is RAG?



The Services — Who Does What

Traefik

The front door · port 80
All browser requests go here first

Auth

port 3001
Login · Register · JWT

AI Service

port 3002
Chat · Images · Videos

RAG

port 3003
PDF upload · Search textbooks

Analytics

port 3004
Track usage · Dashboards

Ollama

native Mac · port 11434
The AI brain · runs LLM + embeddings

ChromaDB

Docker · port 8000
Stores textbook vectors

Postgres

Docker · port 5432
Main database · all services

3 rules to understand first:

- The **browser never talks directly to a service** — every request goes through **Traefik** first, which routes it to the right service.
- **Services talk to each other directly** — e.g. AI Service calls RAG directly without going through Traefik.
- **Ollama runs natively on your Mac** (not in Docker) so it can use the GPU. Services reach it via a special address `host.docker.internal:11434`



Flow 1 — Logging In

All users

1

User types email + password and clicks Login

Browser sends this to **Traefik** → **Auth Service**

2

Auth Service looks up the user in the database **Postgres**

Finds the user by email · checks if the password matches the stored hash using bcrypt · if wrong password → returns error 401

3

Special step for Parents only **Postgres**

Also fetches the list of their linked children (studentIds) from the database · this gets baked into the token

4

Auth Service creates a JWT token **Auth**

Token contains: userId · role (student / teacher / parent) · schoolId · studentIds (parents only) · expires in 24 hours

5

Token is saved as a cookie in the browser

It's an **HttpOnly cookie** — meaning JavaScript cannot read it (more secure) · browser sends it automatically with every future request · user doesn't see it



User is now logged in · browser redirects to their dashboard based on role

Student → chat page · Teacher → class dashboard · Parent → child progress page

Important: Login is rate limited to 10 attempts per minute. If someone tries 11 times → blocked for a minute. This prevents brute force attacks.

Also: Teachers cannot register through the app. Teacher accounts are created only via the seed script.



Flow 2 — Student Chats with the AI Tutor

Student only

WHAT THE STUDENT DOES

Picks a subject (Science / Math / English) → types a question like "What is photosynthesis?" → sees words appearing on screen one by one as the AI responds

1

Student picks a subject and starts a chat session AI Service

Browser calls AI Service with the subject (e.g. "Science") · AI Service creates a session in the database and returns a sessionId · browser saves this sessionId

2

Student types a message and sends it AI Service

Browser sends: the message text + the sessionId to AI Service · AI Service checks: is this a student? Is the message under 500 characters? Does this session exist? If any check fails → error

3

AI Service loads the conversation history Postgres

Fetches the last 10 messages from this session · this gives the AI context of what was already discussed

4

AI Service asks RAG: "Find relevant textbook content for this question" RAG Service

Sends: the student's question + schoolId + subject to RAG Service · RAG converts the question into a vector (a list of numbers that represent meaning) using Ollama · then searches ChromaDB for the 5 most similar paragraphs from NCERT PDFs · returns those 5 chunks of text back to AI Service

5

AI Service builds the prompt for Ollama AI Service

Combines: the system instructions ("You are Roognis, answer only from the context below") + the 5 NCERT paragraphs + the last 10 messages of conversation history + the student's new question

6

AI Service opens an SSE stream to the browser and calls Ollama Ollama

SSE (Server-Sent Events) = a live connection where the server can keep sending data to the browser. Ollama sends back one word at a time → AI Service immediately forwards each word to the browser → student sees words appearing live, just like ChatGPT

7

Once complete, AI Service saves the conversation Postgres + records analytics Analytics

Saves both the student's message and the AI's full response to the database · also fires an event to Analytics (fire-and-forget: if Analytics is down, chat still works fine)



Student sees the full answer on screen · based on actual NCERT content · not made up

What is SSE? Normally a browser asks → server replies → connection closes. SSE keeps the connection open so the server can keep sending data. This is how the words appear one by one instead of all at once after a long wait.



Flow 3 — Student Requests an Educational Image

Student only

WHAT THE STUDENT DOES

Types something like "show me a diagram of photosynthesis" → sees a loading spinner → image appears after a few seconds

1

Student requests an image · AI Service creates a job AI Service

AI Service immediately saves a job to the database with status "queued" · returns a jobId to the browser right away · does NOT wait for the image — returns instantly

2

Background worker processes the job Background

While the browser shows a spinner, a background worker matches keywords in the prompt to a pre-generated image stored in seed-data/images/ · updates the job status to "done" with the image URL

3

Browser polls for status every 3 seconds AI Service

Browser keeps asking: "Is my image ready?" every 3 seconds · when status = "done", the response includes the imageUrl · browser stops polling and displays the image



Student sees the educational image

Images are pre-generated with Stable Diffusion before the demo · looks instant to the user

Timeout protection: A cron job runs every 2 minutes. If any image job is stuck in "processing" for more than 5 minutes (e.g. if something crashed), it automatically marks that job as "failed" so the user's spinner doesn't spin forever.



Flow 4 — Student Watches a Video

Student only

WHAT THE STUDENT DOES

Sees a list of 5 video topics → picks one → video plays in the browser

1

Browser loads the list of available topics AI Service

AI Service returns a hardcoded list of 5 topics: photosynthesis, fractions, water-cycle, parts-of-speech, solar-system · browser shows these as clickable options

2

Student picks a topic → browser fetches the video AI Service

AI Service serves the MP4 file from seed-data/videos/ · the video plays directly in the browser using a standard HTML video player

✓

Video plays · no AI involved · just serving a file**Flow 5 — Student Rates an AI Response**Student only

WHAT THE STUDENT DOES

After an AI answer, sees a 1–5 star rating widget → taps a rating (optionally adds a comment) → done

1

Browser sends the rating to AI Service AI Service

Sends: sessionId + messageId + rating (1–5) + optional comment

2

AI Service saves the feedback Postgres

Stores the rating against the specific message · also fires an event to Analytics so it can be counted

✓

Rating saved · if student keeps giving low ratings → teacher will be alerted on their dashboard**Flow 6 — Teacher Views Class Dashboard**Teacher only

WHAT THE TEACHER SEES

A table showing all students in their class · each row has: name, AI usage count, attendance %, test scores, average feedback rating · students who need attention are highlighted in red

1

Browser loads the dashboard Analytics

Analytics Service reads the teacherId from the JWT token · uses it to look up which students are in this teacher's class (from class_assignments table)

2

Analytics Service queries stats for each student Postgres

For each student: counts chat sessions in last 7 days · calculates attendance percentage · gets test scores · gets average feedback rating

3

Applies the intervention rule **Analytics**

A student gets flagged as "needs help" if EITHER: their average rating is below 3 out of 5 · OR they've had fewer than 3 chat sessions in the last 7 days (meaning they're not using the tutor)



Teacher sees full class overview with red flags on students who need attention

Teacher also has:

- POST /score — enter a test score for a student (only works if student is in their class)
- POST /attendance — mark a student present or absent for a date
- POST /class/assign — add a student to their class
- GET /interventions — see only the flagged students (useful for a notification badge)

**Flow 7 — Parent Views Child's Progress****Parent only**

WHAT THE PARENT SEES

Dropdown to pick which child (if they have multiple) → that child's scores, attendance %, and AI usage summary

1

Frontend shows a child picker from the JWT token

When the parent logged in, their JWT was stamped with the list of their children's IDs · frontend uses this to show a dropdown like "Arjun / Priya" · no extra API call needed for this step

2

Parent picks a child → browser loads that child's data **Analytics**

Sends: GET /api/analytics/parent/dashboard?studentId=arjun-uuid

3

Security check — is this actually your child? **Analytics**

Analytics checks: is the studentId in the request inside the parent's studentIds list from the JWT? · If not → 403 Forbidden · This prevents Parent A from typing Parent B's child's ID to see their data



Parent sees their child's progress · cannot see other children's data

**Flow 8 — Teacher Uploads a Study Material PDF****Teacher only**

WHAT THE TEACHER DOES

Picks a PDF file from their computer · selects the subject · clicks Upload · sees "Processing..." · after a minute or two it shows "Done — 47 chunks embedded"

1

Browser uploads the PDF to RAG Service RAG

Sends the file + subject to RAG Service · if file is bigger than 20MB → rejected immediately with an error · RAG Service saves the file to disk

2

RAG Service responds immediately — does not wait RAG

Returns { documentId, status: "processing" } right away · the teacher doesn't wait staring at a frozen screen · processing happens in the background



BACKGROUND PROCESSING (WHILE TEACHER SEES "PROCESSING...")

B1

Extract text from the PDF

Uses PyMuPDF to read every page and pull out all the text

B2

Split text into chunks

Breaks the text into ~512 word pieces with 50 word overlap between chunks · overlap helps so questions don't fall into the gap between two chunks

B3

Convert each chunk to a vector Ollama

Sends each chunk to Ollama's nomic-embed-text model · gets back a list of 768 numbers (called a vector / embedding) that represents the meaning of that chunk

B4

Store vectors in ChromaDB ChromaDB

Saves each chunk + its vector into a ChromaDB collection named "school_{schoolId}_{subject}" · e.g. "school_demo-uuid_Science" · now this content is searchable by meaning



PDF is now fully embedded · when any student asks a Science question, the AI will find relevant chunks from this PDF and use them in its answer

On first boot: The seed script auto-uploads 5 NCERT PDFs from seed-data/pdfs/ using the demo school ID. So on demo day the AI already knows NCERT content without the teacher doing anything.



What is RAG? — Simple Explanation

For the whole team

X

Without RAG — The Problem

AI answers from general training data · gives generic Wikipedia-level answers · makes things up (hallucination) · not aligned to NCERT curriculum · a student using it for exams gets wrong information

R

Retrieve — Find relevant textbook content

Student asks "What is photosynthesis?" · we convert this question into a vector (768 numbers representing its meaning) · search ChromaDB for the most similar vectors · find the 5 most relevant paragraphs from NCERT Science · these are actual textbook content, not AI guesses

A**Augment — Add context to the prompt**

Take those 5 NCERT paragraphs and add them to the prompt sent to Ollama: "Here is relevant content from the textbook: [paragraphs]. Answer this student's question using ONLY this content."

G**Generate — AI answers from real content**

Ollama generates an answer based on actual NCERT text · cannot hallucinate because it's told to only use the provided context · if the answer isn't in the textbook it says "I don't have information on that yet"

✓**Result: AI tutor that answers exactly from the student's own textbook**

Different schools → different PDFs uploaded → different answers tailored to their curriculum

Why does vector search work? Normal database search matches exact words. Vector search matches meaning. "Chlorophyll" and "green pigment that captures light" have no words in common but a vector search knows they're about the same thing. This is why RAG finds the right content even when the student's phrasing doesn't match the textbook's exact words.